

UNIVERSIDAD CATÓLICA DE CUENCA

**FACULTAD DE INFORMÁTICA, CIENCIAS DE LA
COMPUTACIÓN E INNOVACIÓN TECNOLÓGICA**

**CARRERA DE ROBÓTICA E INTELIGENCIA
ARTIFICIAL**

ASIGNATURA:
MICROCONTROLADORES

TEMA:
SISTEMA INTELIGENTE DE CONTROL DUAL CON ESP32 E
INTERRUPCIONES

ESTUDIANTES:
Richard Cabrera
Medardo Vivanco

DOCENTE:
ING. PABLO BUESTÁN ANDRADE, PhD

FECHA: 7 de mayo de 2026

1. INTRODUCCIÓN

Los sistemas embebidos modernos requieren mecanismos eficientes para responder a eventos externos y señales analógicas en tiempo real. El uso de interrupciones y temporizadores de hardware permite construir aplicaciones reactivas sin depender de retardos bloqueantes. En esta práctica se implementa un sistema inteligente basado en el ESP32 capaz de gestionar entradas analógicas, eventos digitales y control de actuadores mediante una máquina de estados adaptativa.

2. OBJETIVO GENERAL

Implementar un sistema embebido inteligente utilizando el ESP32 que integre lectura analógica, control PWM, interrupciones externas y temporizadores de hardware para el control dinámico de actuadores.

3. OBJETIVOS ESPECÍFICOS

- Implementar lectura analógica mediante potenciómetros usando `analogRead()`.
- Controlar actuadores utilizando `analogWrite()` y `digitalWrite()`.
- Configurar interrupciones externas mediante pulsadores.
- Implementar temporizadores de hardware para tareas periódicas.
- Utilizar comunicación serial para monitoreo del sistema en tiempo real.

4. DISEÑO ESQUEMÁTICO

Para el diseño y validación del sistema se utilizó la herramienta Wokwi, empleando los siguientes componentes:

- Microcontrolador ESP32.
- 2 LEDs con sus respectivas resistencias de $220\ \Omega$.
- 2 Potenciómetros para la entrada de señales analógicas.
- 2 Pulsadores configurados con interrupciones externas.

5. CÓDIGO FUENTE

```
1 #define PIN_LED1      25
2 #define PIN_LED2      26
3 #define PIN_POT1      34
4 #define PIN_POT2      35
5 #define PIN_BTN1      18
```

```

6 #define PIN_BTN2      19
7
8 #define LEDC_CHANNEL   0
9 #define LEDC_FREQ_HZ   5000
10 #define LEDC_BITS      8
11
12 volatile bool timerFlag = false;
13 volatile bool estadoSistema = true;
14 volatile int modo = 1;
15 volatile unsigned long lastBtn1 = 0;
16 volatile unsigned long lastBtn2 = 0;
17 #define DEBOUNCE_US 200000UL
18 volatile bool estadoLed2 = false;
19
20 hw_timer_t* timer = NULL;
21
22 void IRAM_ATTR onTimer() {
23     timerFlag = true;
24 }
25 void IRAM_ATTR isrBtn1() {
26     unsigned long ahora = micros();
27     if (ahora - lastBtn1 > DEBOUNCE_US) {
28         lastBtn1 = ahora;
29         if (estadoSistema) {
30             modo = (modo == 1) ? 2 : 1;
31         }
32     }
33 }
34 void IRAM_ATTR isrBtn2() {
35     unsigned long ahora = micros();
36     if (ahora - lastBtn2 > DEBOUNCE_US) {
37         lastBtn2 = ahora;
38         estadoSistema = !estadoSistema;
39     }
40 }
41
42 void configurarTimer(uint32_t freqHz) {
43     if (timer != NULL) {
44         timerEnd(timer);
45         timer = NULL;
46     }
47
48     timer = timerBegin(1000000);
49
50     timerAttachInterrupt(timer, &onTimer);
51     uint64_t ticks = 1000000UL / freqHz;
52     timerAlarm(timer, ticks, true, 0);
53 }
54
55 void setup() {
56     Serial.begin(115200);
57     Serial.println("=== SISTEMA ESP32 INICIADO ===");
58     pinMode(PIN_LED1, OUTPUT);
59     pinMode(PIN_LED2, OUTPUT);
60     pinMode(PIN_BTN1, INPUT_PULLUP);
61     pinMode(PIN_BTN2, INPUT_PULLUP);
62     ledcAttach(PIN_LED1, LEDC_FREQ_HZ, LEDC_BITS);
63     attachInterrupt(digitalPinToInterrupt(PIN_BTN1), isrBtn1, FALLING

```

```

    );
64   attachInterrupt(digitalPinToInterrupt(PIN_BTN2), isrBtn2, FALLING
    );
65   configurarTimer(2);
66   Serial.println("Hardware Timer configurado: 2 Hz");
67   Serial.println("Modo inicial: 1 (Control PWM)");
68 }
69
70 void loop() {
71   if (!estadoSistema) {
72     ledcWrite(PIN_LED1, 0);
73     digitalWrite(PIN_LED2, LOW);
74     Serial.println("SISTEMA DESACTIVADO");
75     while (!estadoSistema) {
76     }
77     Serial.println("SISTEMA REACTIVADO");
78     return;
79   }
80
81   if (modo == 1) {
82     digitalWrite(PIN_LED2, LOW);
83     int valorADC = analogRead(PIN_POT1);
84     int valorPWM = map(valorADC, 0, 4095, 0, 255);
85     ledcWrite(PIN_LED1, valorPWM);
86     Serial.print("MODO 1 - ADC: ");
87     Serial.print(valorADC);
88     Serial.print("PWM: ");
89     Serial.println(valorPWM);
90   }
91
92   else if (modo == 2) {
93     ledcWrite(PIN_LED1, 0);
94     int valorPot2 = analogRead(PIN_POT2);
95     int freqParpadeo = map(valorPot2, 0, 4095, 20, 1);
96     static int freqAnterior = 0;
97     if (freqParpadeo != freqAnterior) {
98       configurarTimer(freqParpadeo);
99       freqAnterior = freqParpadeo;
100    }
101    if (timerFlag) {
102      timerFlag = false;
103      estadoLed2 = !estadoLed2;
104      digitalWrite(PIN_LED2, estadoLed2 ? HIGH : LOW);
105
106      Serial.print("MODO 2 / ADC: ");
107      Serial.print(valorPot2);
108      Serial.print("Frecuencia: ");
109      Serial.print(freqParpadeo);
110      Serial.print("Hz/LED2: ");
111      Serial.println(estadoLed2 ? "ON" : "OFF");
112    }
113  }
114 }

```

6. FOTOGRAFÍAS Y EVIDENCIAS

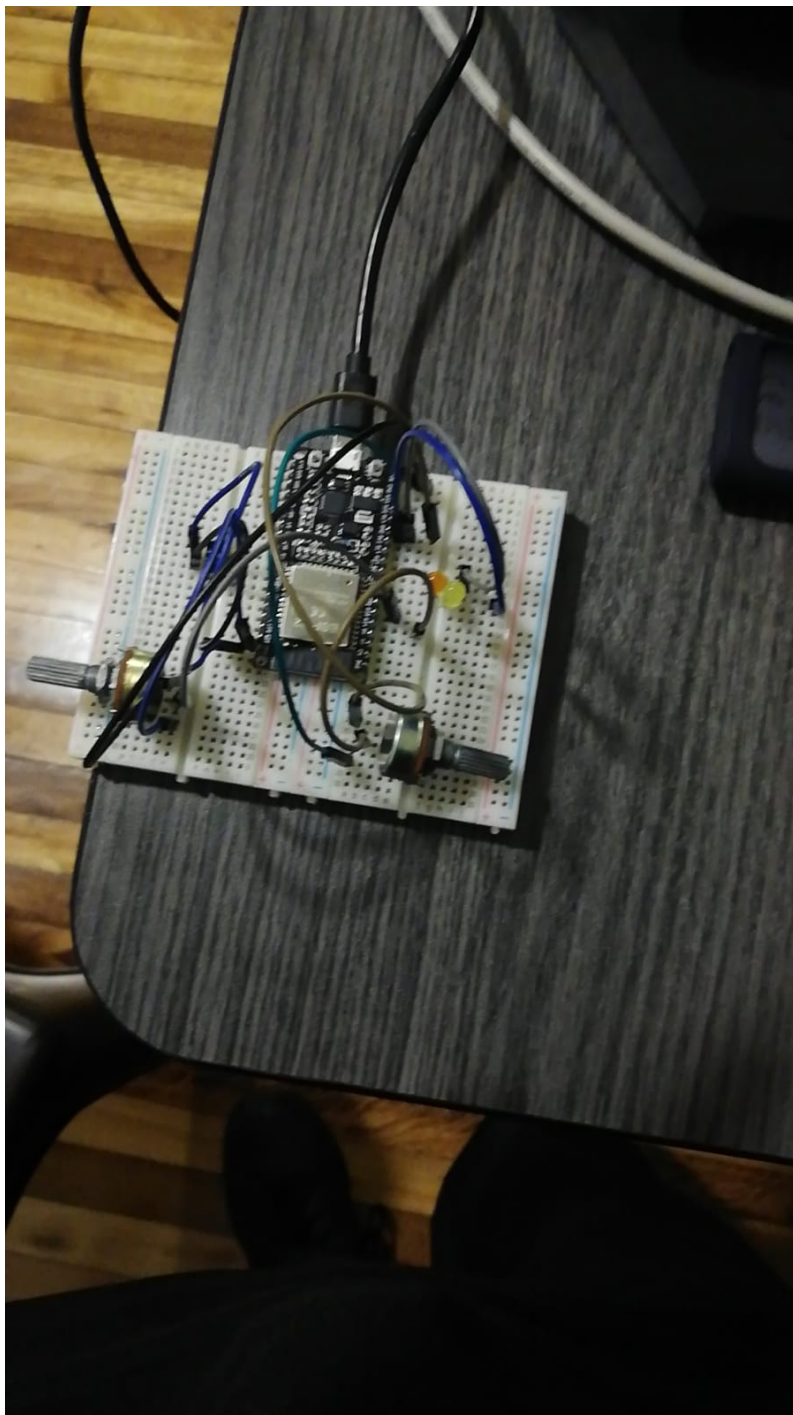


Figura 1: Figura 1: Implementación del circuito dual con ESP32 en físico.

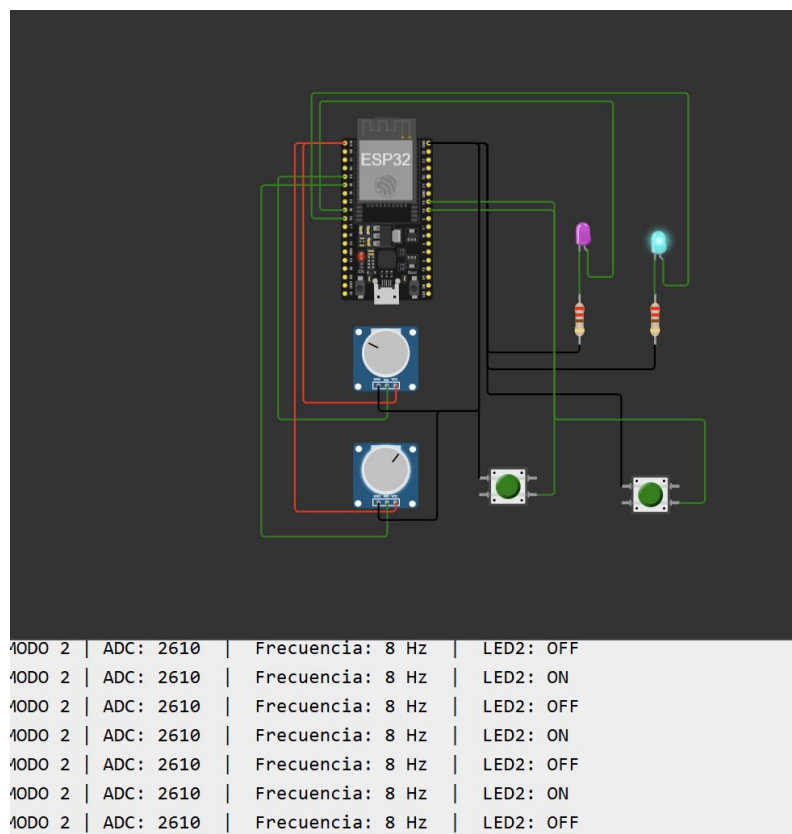


Figura 2: Figura 2: Simulación del sistema en funcionamiento junto con los registros del Monitor Serial.